# INF712 - Computerunterstützte Statistik

**26-27 WS**

08/10/2026

## Inhaltsverzeichnis


**1 Vorlesung 01-01: Statistik und die Sprache R** **1**

1.1 Formel . . . 3
1.2 Quellcode . . . 3
1.3 Erklärung . . . 5
1.4 Überblick . . . 6
1.5 Numerische Ablaufverfolgung . . . 15


# 1 Vorlesung 01-01: Statistik und die Sprache R

**Frage:** Was muss eine Programmierungsprache leisten, um gut für Statistik geeignet zu sein?

**1. Statistische Datenstrukturen:** Verwendet die Sprache als Datastruktur Vektoren, Faktoren (für kategorische Varieblen wie männlich/weiblich) und Datenrahmen? Werden fehlende Daten berücksichtigt?

**2. Breite und Korrektheit statistischer Algorithmen:** Wurden die Methoden von Personen implementiert, die die Statistik verstehen, und wurden sie fachlich begutachtet?

**3. Effizienz bei großen Datenmengen:** Verwendet die Sprache vektorisierte und matrixbasierte Operationen anstatt naiver Schleifen.

**Beispiel: Addition von mehreren Zahlen mit 1**

```
x=c(2,3,8)
y=x+1
print(y)
#> [1] 3 4 9
```

**4. Numerische Stabilität:** Wie werden Rundungsfehler unter Kontrolle gehalten, sodass sie sich nicht in große Gesamtfehlern addieren? Wie werden Überlauf (overflow, im Compter nicht representierbare, ganz große Zahlen tauchen auf) und Unterlauf (underflow, sehr kleine Zahlen werden auf Null gerundet) kontrolliert?

### Beispiel: Mittelwertsberechnung

Mathematische Formulierung des arithmetischen Mittelwerts von $n$ Zahlen ist gegeben als:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i. \tag{1}$$

Wir betrachten den Mittelwert von $10^7$ Zahlen in der Form:

$$x_i = 10^{12} + i, \qquad i = 1, 2, \cdots, 10^7$$

Mithilfe von

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2},$$

kann der echte Mittelwert als

$$\bar{x} = \frac{1}{10^7}\left[(10^{12}+1) + (10^{12}+2) + \cdots (10^{12}+10^7)\right] = \frac{1}{10^7}\left[10^7 \cdot 10^{12} + \sum_{k=1}^{10^7} k\right] \tag{2}$$

$$= 10^{12} + \frac{1}{10^7} \cdot \frac{10^7(10^7+1)}{2} = 1000005000000.5 \tag{3}$$

berechnet werden.

Mithilfe vom Computer berechnen wir diesen in 3 verschiedenen Arten:

**1)** Die Summe wird mit einer Schleife berechnet:

```
    n<-1e7
    x<-1e12 + seq_len(n)                                    ①

    loopsum <- 0
    for (i in seq_along(x))                                 ②
      loopsum <- loopsum + x[i]
    xbar <- loopsum / n
    print(sprintf("Ergebnis mit Schleife: %.10f", xbar))

#> [1] "Ergebnis mit Schleife: 1000005000000.0401611328"
```

① Die Sequenz wird erzeugt
② In jeder Iteration der Schleife wird eine neue Zahl addiert.

**2)** Die Summe wird mit der eingebauten Summenfunktion **sum()** berechnet:

```
    n<-1e7
    x<-1e12 + seq_len(n)

  xbar=sum(x)/n;
  print(sprintf("Ergebnis mit sum(x): %.10f", xbar))
#> [1] "Ergebnis mit sum(x): 1000005000000.4998779297"
```

**3)** Der Mittelwert wird direkt mit der eingebauten Mittelwertsfunktion **mean()** berechnet:

```
    n<-1e7
    x<-1e12 + seq_len(n)

  xbar=mean(x);
  print(sprintf("Ergebnis mit mean(x): %.10f", xbar))
#> [1] "Ergebnis mit mean(x): 1000005000000.5000000000"
```

Man beobachtet, dass man für jede Methode eine andere Antwort bekommt, wobei die Funktion **mean()** die genaueste Antwort liefert. Der Grund dafür ist es, dass diese Funktionen die mathematische Formulierung nicht direkt anwenden, sondern in sich auch Algorithmen für Korrektur der Rundungfehler beinhalten.

> ❗ Vorsicht!
>
> Rechnen am Computer ist nicht die direkte Implemantation der mathematischen Formel.

**Übung: Was geschieht intern in R bei mean(c(2,5,6))?**

Analysieren Sie die Quellencode (source code) der Mittelwertsfunktion **mean()** in R, der auch einen C-Code beinhaltet. Auf der Webseite https://github.com/wch/r-source können Sie Ressourcen finden, mit denen man die interne Funktionsweise von R verstehen kann. In /src/main/summary.c versuchen Sie die Mittelwertsfunktionen zu bestimmen.

## 1.1 Formel

$$\bar{x} = \frac{1}{n}\sum_{k=1}^{n} x_k$$

## 1.2 Quellcode

https://github.com/wch/r-source/blob/trunk/src/main/summary.c (Zeile 479-518: real_mean())

```
static R_INLINE SEXP real_mean(SEXP x)
{
    R_xlen_t n = XLENGTH(x);
    LDOUBLE s = 0.0;
    ITERATE_BY_REGION(x, dx, i, nbatch, double, REAL, {
        for (R_xlen_t k = 0; k < nbatch; k++)
        s += dx[k];
    });
    bool finite_s = R_FINITE((double) s) != 0; //isfinite returns non-zero
    if (finite_s) {
    s /= n;
    DbgP3("real_mean(): n=%g, s=%g\n", (double)n, s);
    } else { // infinite s, maybe just overflowed; try to use smaller terms:
    DbgP3("real_mean(): n=%g, infinite s=%g -- try again: ", (double)n, s);
    s = 0.;
    ITERATE_BY_REGION(x, dx, i, nbatch, double, REAL, {
        for (R_xlen_t k = 0; k < nbatch; k++)
            s += dx[k]/n;
        });
    DbgP2(" --> new s=%g\n", s);
    }
    if (finite_s && R_FINITE((double) s)) {
    LDOUBLE t = 0.0;
    ITERATE_BY_REGION(x, dx, i, nbatch, double, REAL, {
        for (R_xlen_t k = 0; k < nbatch; k++)
            t += (dx[k] - s);
        });
    s += t/n;
    }
    else if (R_FINITE((double) s)) { // was infinite: more careful
    LDOUBLE t = 0.0;
    ITERATE_BY_REGION(x, dx, i, nbatch, double, REAL, {
        for (R_xlen_t k = 0; k < nbatch; k++)
            t += (dx[k] - s)/n;
        });
    DbgP2(" s = s + t, t=%g\n", t);
    s += t;
    }
```

```
    return ScalarReal((double) s);
}
```

### 1.3 Erklärung

Wenn Sie

```
mean(c(2, 5, 6))
```

ausführen, durchläuft R mehrere Ebenen. Die sichtbare R-Funktion berechnet den Mittelwert nicht direkt; die eigentliche numerische Berechnung erfolgt im C-Code in der Datei `summary.c`.

```
mean(c(2, 5, 6))


Erzeugen des numerischen Vektors c(2, 5, 6)


mean()                  S3-Generic


mean.default()


.Internal(mean(x))


do_summary()            C-Funktion


real_mean()             C-Funktion


[1] 4.333333
```

Der R-Quellcode zeigt, dass `mean()` über ein R-seitiges Generic weitergeleitet wird und die interne Implementierung schließlich als Spezialfall von `mean.default()` aufgerufen wird.

## 1.4 Überblick

**Schritt 1: R analysiert den Ausdruck**

R analysiert zunächst

```
mean(c(2, 5, 6))
```

als verschachtelten Funktionsaufruf:

```
mean(
    c(2, 5, 6)
)
```

Bevor `mean()` aufgerufen werden kann, muss das Argument normalerweise ausgewertet werden. Daher wertet R zunächst

```
c(2, 5, 6)
```

aus.

**Schritt 2: R erzeugt den Vektor**

Die Literale

```
2
5
6
```

sind gewöhnliche numerische Konstanten in R. Solche Konstanten werden standardmäßig als Gleitkommazahlen mit doppelter Genauigkeit gespeichert.
Daher ist das Ergebnis von

```
c(2, 5, 6)
```

konzeptionell:

```
c(2.0, 5.0, 6.0)
```

Der interne R-Typ lautet:

```
REALSXP
```

Sie können dies auf R-Ebene überprüfen:

```
typeof(c(2, 5, 6))
```

Ergebnis:

```
[1] "double"
```

Diese Unterscheidung ist wichtig. Hätten Sie

```
c(2L, 5L, 6L)
```

geschrieben, hätte der Vektor den internen Typ `INTSXP`, und R würde `integer_mean()` statt `real_mean()` verwenden.

**Schritt 3: R sucht die Funktion `mean`**

R findet die Funktion `mean` im Basispaket.
Auf R-Ebene ist `mean()` ein S3-Generic, konzeptionell definiert als:

```
mean <- function(x, ...)
    UseMethod("mean")
```

Das bedeutet, dass `mean()` selbst nicht den numerischen Algorithmus enthält. Stattdessen fragt R zunächst:

> Welche `mean`-Methode soll für dieses Objekt verwendet werden?

**Schritt 4: Die S3-Methodenauswahl wählt `mean.default()`**

Der Vektor

```
c(2, 5, 6)
```

besitzt keine besondere S3-Klasse:

```
class(c(2, 5, 6))
```

liefert:

```
[1] "numeric"
```

Es wird hier keine spezielle Methode wie `mean.numeric()` verwendet. Daher wählt die S3-Methodenauswahl:

```
mean.default()
```

Konzeptionell wird der Aufruf zu:

```
mean.default(c(2, 5, 6))
```

Die Standardargumente sind:

```
trim = 0
na.rm = FALSE
```

Der effektive Aufruf lautet somit:

```
mean.default(
    x = c(2, 5, 6),
    trim = 0,
    na.rm = FALSE
)
```

**i** Schritt 5: `mean.default()` überprüft die Eingabe

`mean.default()` überprüft zunächst, ob die Eingabe einen geeigneten Typ besitzt.
Unser Objekt ist ein numerischer Vektor und besteht daher die Prüfung.
Da

```
na.rm = FALSE
```

gilt, entfernt R keine fehlenden Werte. In diesem Beispiel spielt das keine Rolle, weil der Vektor keine `NA`-Werte enthält.
Da

```
trim = 0
```

gilt, wird kein getrimmter Mittelwert berechnet. R fährt daher mit der gewöhnlichen internen Mittelwertberechnung fort.
Der entscheidende Aufruf auf R-Ebene lautet:

```
.Internal(mean(x))
```

An dieser Stelle wechselt die Ausführung vom gewöhnlichen R-Code zur C-Implementierung von R.

**Schritt 6: `.Internal(mean(x))` gelangt zu `do_summary()`**

Die interne Operation wird in `src/main/summary.c` behandelt.
Die relevante C-Funktion heißt `do_summary()`. Sie wird für mehrere zusammenfassende Operationen verwendet, die intern durch einen Operationscode unterschieden werden:

```
0 = sum
1 = mean
2 = min
3 = max
4 = prod
```

Für den aktuellen Aufruf gilt:

```
op = 1
```

Das bedeutet:

```
Mittelwert berechnen
```

**Schritt 7: R überprüft den internen Vektortyp**

Der interne C-Code untersucht:

```
TYPEOF(x)
```

Für unseren Vektor gilt:

```
TYPEOF(x) == REALSXP
```

weil `c(2, 5, 6)` ein Vektor vom Typ `double` ist.
Daher ruft R

```
real_mean(x)
```

auf.
Dies ist die Funktion aus `summary.c`.

**Schritt 8: `real_mean()` bestimmt die Vektorlänge**

Die erste Zeile lautet:

```
R_xlen_t n = XLENGTH(x);
```

Der Vektor enthält drei Elemente. Daher gilt:

```
n = 3
```

`R_xlen_t` ist ein ganzzahliger Typ, den R zur Speicherung von Vektorlängen verwendet, einschließlich der Längen sehr großer Vektoren.

**Schritt 9: R initialisiert den Akkumulator**

Die nächste Zeile lautet:

```
LDOUBLE s = 0.0;
```

Damit gilt zunächst:

```
s = 0
```

`LDOUBLE` ist der von R verwendete Akkumulationstyp. Abhängig von Plattform und Kompilierung kann er eine größere Genauigkeit oder einen größeren Zahlenbereich als ein gewöhnlicher `double` besitzen.
Die Elemente des Vektors selbst bleiben gewöhnliche Gleitkommazahlen mit doppelter Genauigkeit.

**Schritt 10: R führt den ersten Summationsdurchlauf aus**

Der folgende Abschnitt durchläuft den Vektor:

```
ITERATE_BY_REGION(x, dx, i, nbatch, double, REAL, {
    for (R_xlen_t k = 0; k < nbatch; k++)
        s += dx[k];
});
```

Für einen einfachen gewöhnlichen Vektor kann man dies näherungsweise so verstehen:

```
for (k = 0; k < n; k++)
    s += x[k];
```

Die Berechnung verläuft wie folgt.
Anfangswert:

$$s = 0$$

Nach dem ersten Element:

$$s = 0 + 2 = 2$$

Nach dem zweiten Element:

$$s = 2 + 5 = 7$$

Nach dem dritten Element:

$$s = 7 + 6 = 13$$

Daher gilt:

```
s = 13
```

`ITERATE_BY_REGION` ermöglicht es, denselben Code effizient sowohl für gewöhnliche Vektoren als auch für alternative Darstellungen wie ALTREP-Objekte zu verwenden. Für diesen kleinen Vektor verhält sich das Makro konzeptionell wie eine normale Schleife.

**i** Schritt 11: R überprüft, ob die Summe endlich ist

R wertet aus:

```
bool finite_s = R_FINITE((double) s) != 0;
```

Da

```
s = 13
```

endlich ist, gilt:

```
finite_s = true
```

Diese Prüfung ist notwendig, weil die Summe einzeln endlicher Werte unter Umständen zu positiver oder negativer Unendlichkeit überlaufen kann.
In diesem Beispiel geschieht das nicht.

**Schritt 12: R berechnet den vorläufigen Mittelwert**

Da die Summe endlich ist, führt R aus:

```
s /= n;
```

Damit ergibt sich:

$$s = \frac{13}{3}$$

oder näherungsweise:

```
s = 4.333333333333333
```

Dies ist der vorläufige Mittelwert, den wir mit

$$\bar{x}_0 = \frac{13}{3}$$

bezeichnen können.
Eine naive Implementierung könnte an dieser Stelle einfach `s` zurückgeben. R führt jedoch einen zweiten Durchlauf aus, um die numerische Genauigkeit zu verbessern.

**Schritt 13: R initialisiert die Korrektur**

Da sowohl die ursprüngliche Summe als auch der vorläufige Mittelwert endlich sind, gelangt R in den folgenden Block:

```
if (finite_s && R_FINITE((double) s)) {
    LDOUBLE t = 0.0;
    ...
}
```

Der Korrekturakkumulator beginnt mit:

```
t = 0
```

**Schritt 14: R berechnet die Residuen**

R führt für jedes Vektorelement aus:

```
t += dx[k] - s;
```

Konzeptionell berechnet R:

$$t = (2 - s) + (5 - s) + (6 - s).$$

Mit dem exakten mathematischen Wert (s=13/3) erhält man:

$$2 - \frac{13}{3} = -\frac{7}{3},$$

$$5 - \frac{13}{3} = \frac{2}{3},$$

$$6 - \frac{13}{3} = \frac{5}{3}.$$

Die Summe lautet:

$$t = -\frac{7}{3} + \frac{2}{3} + \frac{5}{3} = 0.$$

In exakter Arithmetik ist die Summe der Residuen null, weil (s) der arithmetische Mittelwert ist.
In Gleitkommaarithmetik kann `t` jedoch ein sehr kleiner Wert ungleich null sein, weil (13/3) im Binärsystem nicht exakt dargestellt werden kann. Der genaue Wert kann von Plattform, Compiler und der durch `LDOUBLE` bereitgestellten Genauigkeit abhängen.
Intern erhält R daher entweder:

```
t = 0
```

oder ein sehr kleines Rundungsresiduum nahe null.

**i** Schritt 15: R korrigiert den vorläufigen Mittelwert

R wendet an:

```
s += t/n;
```

Mathematisch:

$$s \leftarrow s + \frac{t}{3}.$$

Da (t) null oder extrem nahe null ist, bleibt das Ergebnis näherungsweise:

$$s = 4.333333333333333.$$

Der Zweck dieses zweiten Durchlaufs wird bei numerisch schwierigen Daten wesentlich wichtiger, insbesondere wenn die Werte einen großen gemeinsamen Offset und nur kleine Unterschiede besitzen.
Der verwendete Algorithmus lautet:

$$\bar{x} = \bar{x}_0 + \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x}_0).$$

Für dieses einfache Beispiel verändert die Korrektur das sichtbare Ergebnis nicht.

**i** Schritt 16: R erzeugt den Rückgabewert

Die letzte C-Anweisung lautet:

```
return ScalarReal((double) s);
```

Dabei geschehen mehrere Dinge:

1. `s` wird von `LDOUBLE` in einen gewöhnlichen C-Typ `double` umgewandelt.
2. `ScalarReal()` erzeugt einen numerischen R-Vektor der Länge eins.
3. Dieser Vektor enthält den berechneten Mittelwert.

Das interne Ergebnis entspricht daher:

```
c(4.333333333333333)
```

Es handelt sich um einen Vektor der Länge eins, weil selbst ein scheinbarer Skalar in R als Vektor mit einem Element dargestellt wird.

**i** Schritt 17: R gibt das Ergebnis durch die vorherigen Ebenen zurück

Das Ergebnis wird über die folgenden Ebenen zurückgegeben:

```
real_mean()
    ↓
do_summary()
    ↓
.Internal(mean(x))
    ↓
mean.default()
    ↓
mean()
```

Das endgültige R-Objekt besitzt:

```
typeof(mean(c(2, 5, 6)))
```

```
[1] "double"
```

und:

```
length(mean(c(2, 5, 6)))
```

```
[1] 1
```

Schritt 18: R gibt das Ergebnis in der Konsole aus

In der interaktiven Konsole zeigt die automatische Ausgabe von R:

```
[1] 4.333333
```

Dies bedeutet **nicht**, dass R intern nur sechs Nachkommastellen gespeichert hat. Der interne Wert besitzt ungefähr 16 signifikante Dezimalstellen an Genauigkeit.
Zum Beispiel:

```
sprintf("%.16f", mean(c(2, 5, 6)))
```

liefert näherungsweise:

```
"4.3333333333333330"
```

Das von R ausgegebene `[1]` bedeutet:

> Das erste angezeigte Element ist Element Nummer 1.

## 1.5 Numerische Ablaufverfolgung

```
Eingabeausdruck:
    mean(c(2, 5, 6))

Erzeugen des Vektors:
    x = c(2.0, 5.0, 6.0)
    TYPEOF(x) = REALSXP
```

```
    length(x) = 3

S3-Methodenauswahl:
    mean(x)
    → mean.default(x)

Interner Aufruf:
    .Internal(mean(x))
    → do_summary(op = 1)
    → real_mean(x)

Erster Durchlauf:
    s = 0
    s = 0 + 2 = 2
    s = 2 + 5 = 7
    s = 7 + 6 = 13

Vorläufiger Mittelwert:
    s = 13 / 3
    s   4.333333333333333

Korrekturdurchlauf:
    t = (2 - s) + (5 - s) + (6 - s)
    t   0

Korrigierter Mittelwert:
    s = s + t/3
    s   4.333333333333333

Rückgabewert:
    ScalarReal((double) s)

Konsolenausgabe:
    [1] 4.333333
```

Der entscheidende Punkt ist, dass R nicht einfach `sum(x)/length(x)` berechnet. Für einen Vektor vom Typ `double` bestimmt `real_mean()` zunächst einen vorläufigen Mittelwert und führt anschließend einen Korrekturdurchlauf über die Residuen aus, um die numerische Genauigkeit zu verbessern.

! R ist die richtige Wahl, weil seine Funktionen von Statistikern und Numerikern entwickelt wurden und dabei wissenschaftlich fundierte Algorithmen zum Einsatz kommen, die Genauigkeit, numerische Stabilität, Effizienz und Reproduzierbarkeit gewährleisten.